

Assignment 3

TEGEL 5

Security Analysis

Introduction

We have implemented authentication mechanisms, input validation systems, SQL injection prevention measures, and access control protocols.

Authentication

To provide safe and secure access to the services of our web application, and to keep unwanted users out, we have implemented authentication by means of account creation. A first-time user has the ability to create an account and can access this account by login based on e-mail address and password. To prevent a single user from having multiple accounts, the required email address needs to be unique.

Next to that, the password is hashed and encrypted using argon2 hashing, to prevent reversing passwords from hashed values. The encrypted values are stored in a secure database, along with the email addresses.

Authorization

The framework uses a role-based access control (RBAC) with three distinct user roles: admin, user, and member. Administrative users possess comprehensive system access including user management and event creation capabilities, while regular users have restricted access to personal information management and event participation functions. Role validation occurs through `SecurityUtils.isValidRole()` method, maintaining a whitelist of acceptable roles and preventing privilege escalation attacks.

Session-based authorization leverages role information stored in user sessions for real-time access control decisions. The `EventManagerServlet` demonstrates proper authorization implementation through role checking before allowing access to administrative functions. Authorization validation includes both positive checks verifying required roles and negative checks ensuring users cannot access functions beyond their authorization level. Administrative function logging ensures comprehensive audit trails for all administrative actions, supporting compliance requirements and security monitoring activities essential for production environments. Cookies have not been implemented yet and will be implemented before the end of week 9.

Threats & Safety Measures

Identified Threats include:

- SQL injection attacks through malicious database query manipulation
- Cross-site scripting (XSS) attacks via malicious script injection
- Session-based attacks including session fixation and hijacking
- Input validation bypass attempts, brute force authentication attacks
- Information disclosure through error messages
- Cross-site request forgery (CSRF) attacks enabling unauthorized actions
- Man-in-the-middle attacks intercepting communications
- Privilege escalation attempts targeting authorization bypass.

Measures Implemented:

- Comprehensive safety measures address identified threats through SQL injection prevention using prepared statements and pattern-based detection of injection attempts
- XSS prevention through input sanitization removing dangerous characters and output encoding
- Input validation implementing length restrictions and format validation across all input vectors
- Session security through secure token generation and timeout implementation
- Error handling preventing information disclosure through generic error messages
- Logging providing comprehensive audit trails for security monitoring and incident response activities

Testing Report

Testing Strategy

Our testing strategy consists of a wide array of test types, like Unit, Integration and User tests to ensure proper functionality of the application. Next to that, we also use User Story tests to ensure our application meets the planned requirements and to make sure we actually produce the product that is needed. All these tests combined will help us reach a guaranteed good result.

Unit Testing

Unit testing was implemented using the JUnit 5 framework, with thorough coverage of the SecurityUtils class. The SecurityUtilsTest class systematically checks critical security functions to ensure they're working as expected.

For email validation, six test cases were included:

- A valid format like user12@gmail.com
- Invalid formats, such as missing domain extensions (user12@gmailcom), missing @ symbols (user12gmail.com), incorrect domain structures (user12@.com), incomplete domains (user12@com), and null inputs.

Role validation was tested across six scenarios as well, covering valid roles (admin, user, member), rejection of invalid roles like guest, and handling of edge cases like empty strings and null values. With all tests succeeding, this establishes a solid foundation for role-based access control.

Location validation focused on three core checks: validating proper location strings, managing null inputs, and defending against SQL injection threats using inputs like "DROP TABLE users;". Every test passed, showing the input sanitization is working effectively.

For username validation, six different scenarios were evaluated:

- Accepting typical usernames like validUser123
- Supporting underscores (e.g., user_name)
- Rejecting empty strings, null values, usernames with spaces (invalid user), and those containing special characters (user!@#).

Integration Testing

Integration testing ensures that different parts of the system work smoothly together. It's divided into two main areas: database integration and servlet integration.

For the database integration tests, we use the `InsertUserTest` class to check whether users can be created successfully with valid input. We also verify that the database connection remains stable under heavy load and that prepared statements execute reliably. All tests passed, which tells us that our data persistence layer is functioning as expected.

On the servlet integration side, we test the full user interaction flow across the `SignupServlet`, `LoginServlet`, and `EventManagerServlet` components. The `SignupServlet` tests cover everything from input validation and password hashing to detecting duplicate users and inserting new ones into the database. The `LoginServlet` is tested for secure authentication: verifying credentials, creating sessions, handling user roles, and managing errors gracefully. Finally, the `EventManagerServlet` tests focus on admin-related tasks—making sure access controls are enforced, event creation works properly, inputs are validated, and security issues are handled correctly.

User Story Testing

User Registration

User testing validates the complete user registration workflow against specified requirements. Test scenarios include successful registration with all required fields properly validated, prevention of duplicate email addresses through database constraint checking, password strength validation according to defined policies, and comprehensive input sanitization to prevent malicious input. Implementation verification confirms email format validation in `SignupServlet`, Argon2 password hashing implementation, duplicate user detection mechanisms and comprehensive input sanitization processes.

All user registration test cases pass successfully, confirming full compliance with user story requirements.

User Authentication

Authentication testing validates user login functionality against security requirements. Test scenarios encompass valid credential authentication with proper password verification, invalid credential rejection with appropriate error messaging, session creation and management with timeout implementation, and role-based redirection to appropriate application areas. Implementation verification in `LoginServlet` confirms proper password verification using Argon2 hash comparison, session timeout implementation set to 30

minutes, and role-based routing distinguishing between admin and regular user access. All authentication test cases achieve successful validation.

Admin Event Managing

Administrative functionality testing validates event management capabilities against administrative user requirements. Test scenarios include admin authorization verification before access to administrative functions, event creation with comprehensive input validation, enforcement of input length limits to prevent buffer overflow attacks, and date validation for event scheduling. Implementation verification in `EventManagerServlet` confirms admin role verification before function access, comprehensive event validation including all required fields, and robust security exception handling. All administrative functionality tests meet validation requirements.

Security Testing

Security testing encompasses SQL injection testing, Cross-Site Scripting (XSS) testing, and input validation testing. SQL injection testing confirms detection and blocking of malicious SQL patterns, effectiveness of prepared statements in preventing injection attacks, and proper input sanitization removing dangerous characters. All SQL injection tests pass, confirming robust protection against database attacks.